

Redspher Technikai Interjú Felkészítő

Sallang nélkül. Telefonon olvasható. A cél: szakmai kérdéseknél technikailag alátámasztható válaszok.

Fókusz

Szerep fókusza: business-oriented Full-Stack Developer, React + TypeScript, backend services, AWS, REST APIs, Git/CI/CD.

Extra jelzés: AI-driven solutions, agent-based architecture, MCP, automation, intelligent workflows.

Plusz: PHP előjöhét legacy integráció vagy karbantartás témában.

Tartalom

1. Technikai térkép a JD alapján
2. TypeScript mélység
3. React technikai mélység
4. Node.js és backend services
5. REST API design
6. AWS architektúra mélység
7. CI/CD, quality és delivery
8. AI, LLM, MCP és agentek
9. Legacy PHP integráció
10. Mini system design: logistics quote workflow
11. Valószínű technikai kérdések
12. Rövid válaszbank
13. Gyakori csapdák
14. 30 perces ismétlő checklist

Interjú mód

Hogyan használd ezt a PDF-et

Ne hosszú szövegeket magolj. Ezek rövid technikai válaszvázak.

Minden kérdésnél ez legyen a sorrend: definíció, gyakorlati ok, implementációs részlet, trade-off vagy hibamód.

Csak akkor beszélj szakmai múltról, ha rákérdeznek. Maga a válasz álljon meg technikailag önmagában.

Senior hangzású válaszszerkezet

1. Tisztázd a követelményt. Mit optimalizálunk: sebesség, költség, megbízhatóság, karbantarthatóság vagy delivery risk?
2. Mondj egy tiszta alapmegoldást. Először egyszerű design, nem a legbonyolultabb.
3. Tedd hozzá a constraint-eket. Adatméret, forgalom, hibamódok, jogosultságok, audit igény.
4. Magyarázd el a trade-offot. Miért ez, mit nem old meg, mikor váltanál másra.
5. Említs observability-t. Logok, metrikák, trace-ek, alarmok, dashboardok.

1. Technikai térkép a JD alapján

Mit fognak valószínűleg tesztelni?

Tudsz-e end-to-end feature-öket építeni úgy, hogy nem bújssz el csak frontend vagy csak backend mögé?

Tudsz-e React/TypeScript minőségről, API designról, AWS deploymentről, reliability-ről és delivery flow-ról értelmesen beszélni?

Tudsz-e AI/agent/MCP témákról pragmatikusan beszélni, nem buzzword szinten?

Tudsz-e business requirement közelében dolgozni úgy, hogy közben a technikai design tiszta maradjon?

Mit kerülj?

Ne magyarázd túl a CV-det, ha nem kérdezik.

Ne csak tool nevekkal válaszolj. Mindig mondd el: miért és hogyan.

Ne add túl az AI agenteket. Mutass határokat: jogosultság, audit, human approval, monitoring.

Ne javasolj microservice-t vagy event-driven architektúrát alpból, csak ha a probléma indokolja.

2. TypeScript mélység

Miért értékes a TypeScript egy full-stack codebase-ben?

Sok integrációs hibát runtime helyett compile time-ra hoz: rossz DTO mezők, invalid state-ek, hiányzó case-ek, unsafe null kezelés.

Az igazi érték nem az, hogy "mindenen van type". Hanem hogy a type-ok leképezik a business state-eket és API contractokat.

Jó válaszcím: strict mode, explicit DTO-k, schema validation boundary-knál, discriminated union state machine-ekhez, és ha lehet generált API type-ok.

Hogyan modelleznél async data loadingot Reactben?

Több boolean helyett discriminated uniont használnék. Így nem jöhet létre lehetetlen állapot, például loading + success + error egyszerre.

```
type LoadState<T> =  
  | { status: "idle" }  
  | { status: "loading" }  
  | { status: "success"; data: T }  
  | { status: "error"; error: string };
```

Senior pont: a UI state modellezze a valódi lifecycle-t. Ettől kevesebb conditional bug lesz, és a render logika exhaustívabb.

Hol nem véd a TypeScript?

Runtime boundary-knál: HTTP input, database row, environment variable, local storage, message queue, third-party API.

Ilyenkor runtime validation kell: Zod, io-ts, Yup, class-validator vagy custom guard. Nem a konkrét tool a lényeg, hanem az untrusted input validálása.

Interjú mondat: "TypeScript protects code I compile; validation protects data I receive."

Mikor hasznosak a genericek?

Akkor, ha a function viselkedése ugyanaz, de az adatalak változik: API response wrapper, table component, form helper, repository method.

Kerülni kell a túl okos generic type-okat, amit csak a szerző ért. Senior kód: type-safe, de olvasható.

```
type ApiResult<T> =  
  | { ok: true; value: T }  
  | { ok: false; error: ApiError };
```

3. React technikai mélység

Mitől renderel újra egy React komponens?

State update, parent re-render, context value change és external store update okozhat renderelést.

A rendering azt jelenti, hogy React meghívja a component functiont, és kiszámolja a következő UI-t. A commit phase-ben kerülnek a DOM módosítások alkalmazásra.

Senior pont: a re-render önmagában nem probléma. A probléma az expensive render, unstable props, unnecessary effect vagy nagy tree update.

Mikor használnál useMemo-t, useCallbackot vagy memo-t?

Expensive calculation vagy memoized childnak átadott stabil prop esetén, nem reflexből mindenhol.

Először az okot kell megtalálni: large list, unstable object/function prop, context update, expensive derived data vagy unnecessary state.

Trade-off: a memoization komplexitást ad és elfedhet design problémákat. Ha lehet, mérni kell React DevTools-szal vagy browser profilinggal.

Hogyan strukturálnál egy karbantartható React feature-t?

Válaszd szét: API/data access, domain mapping, state logic, reusable UI, page composition.

Business decision ne kerüljön low-level UI componentbe. A display component legyen könnyen tesztelhető és újrahasználatos.

Jó struktúra: feature folder components, hooks, API client, types, tests és egy kis public index résszel.

Hogyan kezelnéd a formokat és validationt?

Client-side és server-side validation is kell. A client validation UX, a server validation a valódi security boundary.

Typed form values, tiszta error mapping, dirty/touched state, disabled submit pending request alatt, accessible error message-ek.

Komplex formnál a formot adatként érdemes modellezni, és kerülni kell a validation logic szétszórását sok componentbe.

React effects: gyakori hiba?

useEffect használata olyan render data deriválására, ami propsból vagy state-ből simán kiszámolható render közben.

Az effect külső rendszerekkel való szinkronizációra való: network, subscription, timer, imperative API.

Dependency arrayt nem érzésre kell kezelni. Ha loopot okoz, általában a design vagy identity stability a probléma.

4. Node.js és backend services

Hogyan magyaráznád a Node.js event loopot?

A Node.js JavaScriptet az event loopon futtat, és sok drága I/O munkát az underlying systemre vagy worker poolra delegál.

Jó I/O-heavy API-khoz és realtime rendszerekhez. CPU-heavy munkára nem jó a main threaden.

Senior pont: kerülni kell a blocking műveleteket, nagy JSON parsingot hot pathon, sync filesystem callokat és hosszú CPU loopokat request handlerben.

Hogyan designolnál egy backend endpointot?

Először contract: method, path, request DTO, response DTO, error format, authorization, idempotency, pagination/filtering ha kell.

Implementációs rétegek: route/controller -> validation -> authorization -> service/use-case -> repository/integration -> response mapping.

Legyen correlation/request ID-s logging, latency/error rate metrika és értelmes domain error.

Hogyan kezelnéd a backend errorokat?

Külön kell kezelni: programmer error, validation error, domain error, integration error, transient infrastructure error.

Stack trace és internal detail ne menjen kliensnek. Stabil error code és ahol kell human-readable message.

Retry csak transient failure esetén, backoffal és limittel.
Non-idempotent operationt nem szabad vakon retryolni.

Hogyan skáláznál Node.js-t?

Először blocking work eltávolítása és bottleneck mérés. Utána horizontal scaling load balancer mögött, stateless service, external storage/cache és queue background workhoz.

CPU-heavy taskra worker thread, külön service, batch job vagy managed processing service.

Session state legyen externalizálva, ha több instance szolgálja ki ugyanazt az appot.

5. REST API design

Mitől jó egy REST API?

Tiszta resource model, kiszámítható URL-ek, helyes HTTP methodok, értelmes status code-ok, stabil contract, konzisztens error format, jó dokumentáció.

Egyszerű CRUD-ra ne legyen RPC-s endpoint. De nem kell dogmatikusnak lenni: command-like action elfogadható, ha a business action nem sima resource update.

Mindig gondolj ezekre: idempotency, pagination, filtering, sorting, versioning, auth, rate limiting, observability.

Hogyan versionölnél egy API-t?

Breaking change-et kerülni kell, ha lehet: optional field hozzáadása, új endpoint, régi behavior támogatása migration alatt.

Gyakori opciók: URL versioning (/v1), header/content negotiation, vagy additive change-ekkel kompatibilitás.

Breaking: field jelentésének változtatása, field törlése, required field változtatása, error semantics változtatása, enum behavior olyan módosítása, amit a kliensek nem tolerálnak.

Enum value hozzáadása breaking change?

Lehet breaking olyan kliensnél, ami exhaustive enumként kezeli és crash-el unknown value-ra.

Biztonságosabb design: kliensben unknown/default branch, API docban pedig jelezni, hogy enum open vagy closed.

Senior válasz: szerveroldalon additívnek tűnik, de contract-breaking lehet strict klienseknél, ha nincs forward compatibility.

Hogyan designolnál paginationt?

Offset pagination egyszerű, de nagy és változó datasetnél lassú vagy inkonzisztens lehet.

Cursor/keyset pagination jobb nagy rendezett datasethez, mert stabil pozíciót használ, például createdAt + id alapján.

Adj vissza next cursort, limitet, totalt csak ha olcsó és hasznos. Kell determinisztikus sort order.

6. AWS architektúra mélység

Mikor választanál Lambdát és mikor containert?

Lambda jó event-driven vagy spiky workloadra, rövid futásokkal és kisebb operational overhead-del.

Container jobb long-running service-hez, custom runtime-hoz, persistent connectionhöz, kiszámítható magas forgalomhoz vagy nagyobb runtime/networking kontrollhoz.

Trade-off: Lambda csökkenti a server managementet, de van cold start, timeout limit, concurrency és event payload constraint.

Hogyan designolnál reliable async workflow-t?

Queue a producer és worker közé, hogy a frontend/API ne függjön lassú downstream processingtől.

Idempotency key, retry policy, dead-letter queue, poison-message kezelés, monitoring queue age/depth/DLQ size metrikákra.

Status tárolás, hogy a UI mutassa: pending, processing, completed, failed vagy retryable.

Milyen AWS service-ek illenek ehhez a JD-hez?

Frontend hosting: S3 + CloudFront vagy managed platform, attól függően mi a cég setupja.

API/backend: ECS/Fargate vagy Lambda + API Gateway, runtime és traffic pattern alapján.

Data: RDS/PostgreSQL transactional business data-hoz; DynamoDB high-scale key-value/access-pattern-driven adathoz; S3 dokumentumhoz/log exporthoz.

Async: SQS/SNS/EventBridge; observability: CloudWatch logs/metrics/alarms, X-Ray/OpenTelemetry ha használják.

AWS security válaszváz

Least privilege IAM, nincs long-lived secret a kódban, encrypted storage, secrets management, private networking ahol kell, audit logok.

User-facing API-nál: authentication, authorization, input validation, rate limiting, WAF ha publikus, safe error message.

Agent/automation esetén: külön permission tool/action szinten és human approval sensitive operationre.

7. CI/CD, quality és delivery

Mit ellenőrizzen egy jó CI pipeline?

Reproducible install, type-check, lint, format check, unit test, integration/API test, build, security/dependency scan, artifact creation, deployment checks.

Frontendnél: bundle size, accessibility check, smoke test, kritikus E2E flow ha lehet.

Backendnél: contract test, database migration check, health check, rollback plan, environment-specific configuration validation.

Hogyan release-elni biztonságosan?

Kis változások, feature flag, automated test, staged environment, smoke test, observability és rollback capability.

Rizikós változásnál canary vagy blue/green deployment. Data migrationnál backward-compatible expand/contract step-ek.

A code deployment és feature activation ne legyen összekötve, ha a feature kockázatos.

Hogyan kezelni a technical debtet?

Láthatóvá kell tenni és riskhez kötni: delivery slowdown, incident risk, test fragility, onboarding cost vagy business constraint.

Touched code körül érdemes javítani. Nagyobb debtnél kis biztonságos lépések, nem nagy rewrite.

Senior válasz: "I avoid purity rewrites unless the business risk justifies them."

8. AI, LLM, MCP és agentek

Magyarázd el egyszerűen az MCP-t

Az MCP egy protokoll, ami standardizálja, hogy egy AI client hogyan fér hozzá külső kontextushoz és képességekhez szervereken keresztül.

Tipikus building blockok: resources ad kontextust/adatot, tools ad műveleteket, prompts ad újrahasználgató prompt template-eket.

Jó interjú szög: az MCP nem maga az "agent". Inkább integration infrastructure contexthez és actionökhöz.

Mi az AI agent gyakorlati engineering értelemben?

Egy agent egy loop: goal értelmezés, context vizsgálat, tool választás, lépések végrehajtása, eredmény megfigyelése, következő lépés eldöntése.

A nehéz rész nem csak a model call. A nehéz rész: permission, tool boundary, state, retry, audit log, cost, latency, human approval.

Productionben az agent inkább kontrollált workflow legyen, nem korlátlan autonóm black box.

Hogyan tennél biztonságossá egy agentet?

Limitált toolok és permissionök. Read-only és write actionök szétválasztása. Sensitive operationhöz confirmation.

Tool input validation, minden tool call logolása, trace/audit adat, budget/timeout, failed vagy partial execution kezelése.

Private data védelme. Ne küldjünk felesleges contextet a modelnek. Redaction vagy scoping ahol lehet.

Hogyan illene AI egy logistics businessbe?

Jó példák: shipment classification, pricing support, exception handling, customer-service draft, document extraction, anomaly detection, route/ETA assistance, internal knowledge search.

Rossz példa: teljesen autonóm price change guardrail, audit, business approval vagy rollback nélkül.

Senior szög: assistive workflow-val és mérhető business value-val kezdeni, majd csak bizonyított reliability után bővíteni az automatizációt.

9. Legacy PHP integráció

Hogyan integrálnál legacy PHP rendszerrel?

Először ownership, data model, API surface, release process és risk feltérképezése. Nem azonnali rewrite.

Lehetséges utak: API facade, adapter layer, shared database csak last resortként, event-based synchronization, vagy strangler migration.

Legacy behavior köré characterization tesztek kelleneek változtatás előtt. A migration legyen visszafordítható, ahol lehet.

Hogyan cserélnél le fokozatosan legacy kódot?

Strangler pattern: új funkcionalitás új service-be megy, miközben a régi működik tovább.

Egyszerre egy bounded capability mozogjon. Contract maradjon stabil. Monitoringgal össze lehet hasonlítani régi és új viselkedést.

Kerülni kell a big-bang rewrite-ot, hacsak a legacy rendszer nem kicsi, izolált és jól ismert.

10. Mini system design: logistics quote workflow

Feladat

Tervezz web workflow-t, ahol business user shipment quote-ot kér. A rendszer árat számol, eltárolja az eredményt, és opcionálisan AI segít magyarázni vagy támogatni a pricing decisiont.

Egyszerű architektúra

React + TypeScript UI form validationnel és async status kijelzéssel.

Backend REST API validál, authot ellenőriz, quote requestet létrehoz, request ID-t ad vissza.

Worker queue-ból dolgozza fel a quote calculationt, ha lassú vagy külső providerektől függ.

Database tárolja a requestet, quote resultot, statust, audit feldeket és business rule verziót.

Observability: request/correlation ID, latency/error rate, queue depth, failed calculation, price-rule failure metrikák.

Hol fér be az AI?

AI magyarázhatja miért változott az ár, kinyerhet mezőket shipment dokumentumból, javasolhat missing datát vagy support draftot írhat.

AI ne írja felül csendben a pricing rule-okat. Determinisztikus business rule legyen source of truth, hacsak a business nem vállalja explicit a model-based decisiont.

MCP/agent esetén először read-only resource-ok, később limitált toolok, például "create draft explanation" vagy "fetch shipment details".

Említendő trade-offok

Sync calculation egyszerűbb, de rontja az UX-et és reliability-t, ha integrációk lassúak.

Async workflow resilientebb, de kell status handling, idempotency, retry/DLQ és több observability.

Relational DB jó default transactional business data-hoz. Cache vagy specializált store csak akkor kell, ha access pattern indokolja.

11. Valószínű technikai kérdések

React + TypeScript

Hogyan előznéd meg az impossible UI state-eket?

Mikor használsz useMemo/useCallback/memo-t?

Hogyan strukturálsz egy feature foldert?

Hogyan kezeled az API errorokat a UI-ban?

Hogyan tartod karbantarthatónak és accessible-nek a formokat?

Backend + REST

Hogyan designolsz REST endpointot business requirementből?

Hogyan kezeled a validationt, authorizationt és error mappinget?

Hogyan teszed idempotenssé a write operationöket?

Hogyan versionölsz API-t breaking client nélkül?

Hogyan designolsz pagination/filtering/sortingot?

AWS + reliability

Mikor választasz Lambdát és mikor ECS/Fargate-et?

Hogyan designolsz async processinget SQS-szel?

Hogyan monitorozol production API-t?

Mi kerüljön CloudWatch alarmba?

Hogyan kezeled a secreteket és IAM permissionöket?

AI/agents

Mi az MCP és miért hasznos?

Mi a különbség tool, resource és prompt között?

Hogyan adnál AI assistantet business workflow-hoz biztonságosan?

Hogyan akadályoznád meg, hogy egy agent veszélyes műveletet végezzen?

Hogyan méred, hogy egy AI feature hasznos-e?

12. Rövid válaszbank

"How do you turn a business need into a technical solution?"

I first clarify the actual business outcome, users, data, constraints and failure modes. Then I define a small contract: UI flow, API boundary, data model and acceptance criteria. I prefer a simple design that can be measured and extended, with tests and observability from the beginning.

"How do you approach scalability?"

I do not start with complex architecture. I identify the bottleneck: frontend rendering, API latency, database queries, external integrations, or background processing. Then I apply the smallest useful change: indexing, caching, queueing, horizontal scaling, or splitting a service only when the boundary is clear.

"How do you approach reliability?"

I assume dependencies fail. I design timeouts, retries with backoff, idempotency, circuit-breaker style protection where needed, queue-based decoupling, DLQs for failed async work, health checks, alarms and clear recovery paths.

"How do you approach AI features?"

I start with assistive use cases where the model improves speed or quality but does not silently make irreversible decisions. I define the context, allowed tools, permission boundaries, evaluation criteria, logging and human approval for sensitive actions.

13. Gyakori csapdák

Csapda: "Mindenre microservice"

Jobb válasz: először modularitás, tiszta boundary-k, split csak akkor, ha team ownership, scaling, deployment independence vagy fault isolation indokolja.

Csapda: "A TypeScript miatt nincs runtime bug"

Jobb válasz: a TypeScript a compiled codebase-en belül segít, de boundary-knál runtime validation kell.

Csapda: "Reactben mindent memoizálni kell"

Jobb válasz: először mérni kell. Memoization hasznos expensive calculationre vagy memoized childnak adott stabil propokra, de komplexitást ad.

Csapda: "Az AI agent hívhatja az összes céges API-t"

Jobb válasz: csak scoped toolok, read/write permission szétválasztása, tool call log, approval sensitive actionre, adversarial input tesztek.

Csapda: "A retry megoldja a hibákat"

Jobb válasz: retry csak transient failure esetén, limit és backoff mellett, és csak ha a művelet safe vagy idempotens.

14. 30 perces ismétlő checklist

Hívás előtt

TypeScript discriminated union és runtime validation átismétlése.

React rendering, effects, memoization és state structure átismétlése.

REST API design: status code, idempotency, versioning, pagination, error format.

AWS reliability: queue, retry, DLQ, alarm, IAM, secrets, rollback.

MCP alapok: resources, tools, prompts, safety és human approval.

Egy tiszta system design fejben: logistics quote/pricing workflow async processinggel és observability-vel.

Legjobb záró mindset

Engineering judgment legyen a válaszokban: simple first, typed contracts, validated boundaries, observable production behavior, safe AI automation és világos trade-offok.

Forrásjegyzet

A dokumentum szándékosan gyakorlati és interjúorientált, nem a források másolata.

- JD alap: Redspher Full Stack Developer, Contern, Luxembourg: React/TypeScript, backend services, AWS, REST APIs, CI/CD, AI/MCP/agents, PHP plusz.
- React official docs: render/commit, state, effects, memoization.
- TypeScript official handbook: narrowing, discriminated unions, generics.
- Node.js official docs: event loop és blocking work kerülése.
- AWS Well-Architected Framework: operational excellence, reliability, security, observability.
- AWS Lambda/SQS/API Gateway/CloudWatch docs: async processing, DLQ, metrics és logging.
- Model Context Protocol official specification: resources, tools, prompts, security and trust.